계정 관리 및 트랜잭션 처리 분석 문서

개요

Oasis CLI의 계정 관리 및 트랜잭션 처리 시스템은 합의 계층(consensus layer)과 ParaTime 계층 모두에서 안전하고 효율적인 트랜잭션 실행을 지원합니다. 이 문서는 계정 생성, 로딩, 관리 과정과 트랜잭션 생성, 서명, 브로드캐스트 과정을 상세히 분석합니다.

계정 관리 시스템

계정 구조 및 타입

계정 설정 구조 (`config/config.go`)

```
type Account struct {
Description  string  // 계정 설명
Kind  string  // 계정 종류 (file, ledger, test)
Address  string  // 계정 주소
Config  map[string]interface{} // 계정별 설정
}
```

지원되는 계정 타입

1. **파일 기반 계정 (file)**

- 암호화된 키 파일로 저장

- 비밀번호로 보호
- 다양한 암호화 알고리즘 지원

2. **Ledger 하드웨어 지갑 (ledger)**

- 하드웨어 기반 보안
- 디바이스에서 직접 서명
- 개인키가 디바이스를 벗어나지 않음

3. **테스트 계정 (test)**

- 개발 및 테스트 목적
- 미리 정의된 키 쌍 사용
- `test:alice`, `test:dave` 등의 형식

계정 로딩 과정

계정 로딩 함수 (`cmd/common/wallet.go`)

```
func LoadAccount(cfg *config.Config, name string) wallet.Account {
// 1. 테스트 계정 확인
if testName := ParseTestAccountAddress(name); testName != "" {
acc, err := LoadTestAccount(testName)
cobra.CheckErr(err)
return acc
}

// 2. 계정 설정 로드
acfg, err := LoadAccountConfig(cfg, name)
```

```
cobra.CheckErr(err)


// 3. 팩토리 로드
af, err := acfg.LoadFactory()
cobra.CheckErr(err)


// 4. 비밀번호 입력 (필요한 경우)
var passphrase string
if af.RequiresPassphrase() && !answerYes {
fmt.Printf("Unlock your account.\n")
err = survey.AskOne(PromptPassphrase, &passphrase)
cobra.CheckErr(err)
}


// 5. 계정 로드
acc, err := cfg.Wallet.Load(name, passphrase)
cobra.CheckErr(err)


return acc
}
```

계정 로딩 단계별 분석

1. **테스트 계정 처리**

- `test:` 접두사로 시작하는 계정명 확인
- 미리 정의된 테스트 키 사용
- 개발 환경에서 빠른 테스트 지원

2. **계정 설정 검증**

- 지갑에 계정 존재 여부 확인
- 계정 설정 정보 로드
- 오류 시 명확한 에러 메시지 제공

3. **팩토리 패턴 활용**

- 계정 타입별 적절한 팩토리 선택
- 확장 가능한 구조로 새로운 백엔드 지원

4. **보안 처리**

- 비밀번호 필요 시에만 요청
- `--yes` 플래그로 자동화 지원
- 안전한 비밀번호 입력 프롬프트

주소 해석 시스템

주소 해석 함수

```
func ResolveLocalAccountOrAddress(net *configSdk.Network, address string) (*ty

// 1. 지갑 계정명 확인

if acc, ok := config.Global().Wallet.All[address]; ok {

addr := acc.GetAddress()

return &addr, nil, nil

}
```

```
// 2. 주소록 엔트리 확인
if entry, ok := config.Global().AddressBook.All[address]; ok {
addr := entry.GetAddress()
return &addr, entry.GetEthAddress(), nil
}

// 3. 일반 주소 해석
return ResolveAddress(net, address)
}
```

지원되는 주소 형식

1. **계정명**: 지갑에 저장된 계정 이름
2. **주소록 엔트리**: 주소록에 저장된 별칭
3. **네이티브 주소**: Oasis 네이티브 주소 형식
4. **이더리움 주소**: 0x로 시작하는 이더리움 형식
5. **명시적 주소**: 특별한 형식의 주소

- `paratime:sapphire` : ParaTime 주소
- `pool:consensus:common` : 풀 주소
- `test:alice` : 테스트 계정 주소

트랜잭션 처리 시스템

트랜잭션 타입 및 구조

합의 계층 트랜잭션

```
// 서명된 합의 트랜잭션
type SignedTransaction struct {
Signed signature.Signed
}

// 서명되지 않은 합의 트랜잭션
type Transaction struct {
Nonce  uint64
Fee *Fee
Method  string
Body  interface{}
}
```

ParaTime 트랜잭션

```
// 서명된 런타임 트랜잭션
type UnverifiedTransaction struct {
Body []byte
AuthProofs []AuthProof
}
```

```go
// 서명되지 않은 런타임 트랜잭션
type Transaction struct {
Version uint16
Call Call
AuthInfo AuthInfo
}
```

트랜잭션 생성 과정

1. 토큰 전송 트랜잭션 (`cmd/account/transfer.go`)

```go
func transferCmd() {
// 1. 매개변수 파싱
var amount, denom, to string
switch len(args) {
case 2:
amount, to = args[0], args[1]
case 3:
amount, denom, to = args[0], args[1], args[2]
}

// 2. 네트워크 연결
ctx := context.Background()
var conn connection.Connection
```

```
if !txCfg.Offline {
conn, err = connection.Connect(ctx, npa.Network)
cobra.CheckErr(err)
}


// 3. 대상 주소 해석
toAddr, toEthAddr, err := common.ResolveLocalAccountOrAddress(npa.Network, to)
cobra.CheckErr(err)


// 4. 계정 로드
acc := common.LoadAccount(cfg, npa.AccountName)


// 5. 트랜잭션 생성 (합의 계층 vs ParaTime)
switch  npa.ParaTime {
case  nil:
// 합의 계층 전송
innerTx := staking.Transfer{
To: toAddr.ConsensusAddress(),
Amount: *amt,
}
tx := staking.NewTransferTx(0, nil, &innerTx)
default:
// ParaTime 전송
```

```
innerTx := accounts.Transfer{
To: *toAddr,
Amount: *amtBaseUnits,
}
tx := accounts.NewTransferTx(nil, &innerTx)
}
}
```

2. 위임 트랜잭션 (`cmd/account/delegate.go`)

```
func delegateCmd() {
// 합의 계층 위임
case nil:
tx := staking.NewAddEscrowTx(0, nil, &staking.Escrow{
Account: toAddr.ConsensusAddress(),
Amount: *amount,
})
sigTx, err = common.SignConsensusTransaction(ctx, npa, acc, conn, tx)

// ParaTime 위임
default:
tx := consensusaccounts.NewDelegateTx(nil, &consensusaccounts.Delegate{
To: *toAddr,
```

```
Amount: *amountBaseUnits,

})

sigTx, meta, err = common.SignParaTimeTransaction(ctx, npa, acc, conn, tx, &txDe

}
```

트랜잭션 서명 과정

합의 계층 트랜잭션 서명

```
func SignConsensusTransaction(

ctx context.Context,

npa *NPASelection,

account wallet.Account,

conn connection.Connection,

tx *consensusTx.Transaction,

) (interface{}, error) {

// 1. 서명자 검증

signer := account.ConsensusSigner()

if signer == nil {

return nil, fmt.Errorf("account does not support signing consensus transactions

}


// 2. 가스 및 수수료 준비

gas, fee, err := PrepareConsensusTransaction(ctx, npa, signer, conn, tx)
```

```
if err != nil {
return nil, err
}

// 3. 논스 조회 (오프라인 모드가 아닌 경우)
if !txOffline && tx.Nonce == invalidNonce {
account, err := conn.Consensus().Staking().Account(ctx, &staking.OwnerQuery{
Height: consensus.HeightLatest,
Owner: account.Address().ConsensusAddress(),
})
if err != nil {
return nil, fmt.Errorf("failed to query account: %w", err)
}
tx.Nonce = account.General.Nonce
}

// 4. 서명 전 확인
PrintTransactionBeforeSigning(npa, tx)

// 5. 트랜잭션 서명
sigCtx := coreSignature.Context([]byte(fmt.Sprintf("%s for chain %s",
consensusTx.SignatureContext, npa.Network.ChainContext)))
signed, err := coreSignature.SignSigned(signer, sigCtx, tx)
```

```
if err != nil {
return nil, err
}

return &consensusTx.SignedTransaction{Signed: *signed}, nil
}
```

ParaTime 트랜잭션 서명

```
func SignParaTimeTransaction(
ctx context.Context,
npa *NPASelection,
account wallet.Account,
conn connection.Connection,
tx *types.Transaction,
txDetails *signature.TxDetails,
) (interface{}, interface{}, error) {
// 1. 가스 및 수수료 준비
gas, fee, feeDenom, err := PrepareParatimeTransaction(ctx, npa, account, conn, t
if err != nil {
return nil, nil, err
}

// 2. 암호화 처리 (필요한 경우)
```

```
var meta interface{}
if txEncrypted {
pk, err := conn.Runtime(npa.ParaTime).Core.CallDataPublicKey(ctx)
if err != nil {
return nil, nil, fmt.Errorf("failed to get runtime's call data public key: %w",
}

cfg := callformat.EncodeConfig{PublicKey: &pk.PublicKey}
var encCall *types.Call
encCall, meta, err = callformat.EncodeCall(&tx.Call, types.CallFormatEncryptedX2
if err != nil {
return nil, nil, fmt.Errorf("failed to encrypt call: %w", err)
}
tx.Call = *encCall
}

// 3. 서명 전 확인
PrintTransactionBeforeSigning(npa, tx)

// 4. 트랜잭션 서명
ts := tx.PrepareForSigning()
sigCtx := &signature.RichContext{
RuntimeID: npa.ParaTime.Namespace(),
```

```
ChainContext: npa.Network.ChainContext,
Base: types.SignatureContextBase,
TxDetails: txDetails,
}
if err := ts.AppendSign(sigCtx, account.Signer()); err != nil {
return nil, nil, fmt.Errorf("failed to sign transaction: %w", err)
}

return ts.UnverifiedTransaction(), meta, nil
}
```

트랜잭션 브로드캐스트 과정

브로드캐스트 함수

```
func BroadcastTransaction(
ctx context.Context,
npa *NPASelection,
conn connection.Connection,
tx interface{},
meta interface{},
result interface{},
) {
```

```
switch  sigTx := tx.(type) {

case *consensusTx.SignedTransaction:

// 합의 계층 트랜잭션 브로드캐스트

fmt.Printf("Broadcasting transaction...\n")

err := conn.Consensus().Core().SubmitTx(ctx, sigTx)

cobra.CheckErr(err)



fmt.Printf("Transaction executed successfully.\n")

fmt.Printf("Transaction hash: %s\n", sigTx.Hash())



case *types.UnverifiedTransaction:

// ParaTime 트랜잭션 브로드캐스트

fmt.Printf("Broadcasting transaction...\n")

rawMeta, err := conn.Runtime(npa.ParaTime).SubmitTxRawMeta(ctx, sigTx)

cobra.CheckErr(err)



// 체크 트랜잭션 오류 확인

if  rawMeta.CheckTxError != nil {

cobra.CheckErr(fmt.Sprintf("Transaction check failed with error: %s",

PrettyErrorHints(ctx, npa, conn, tx, meta, &types.FailedCallResult{

Module: rawMeta.CheckTxError.Module,

Code: rawMeta.CheckTxError.Code,

Message: rawMeta.CheckTxError.Message,
```

```
}))

}


fmt.Printf("Transaction included in block successfully.\n")

fmt.Printf("Round: %d\n", rawMeta.Round)

fmt.Printf("Transaction hash: %s\n", sigTx.Hash())


// 실행 결과 처리

decResult, err := callformat.DecodeResult(&rawMeta.Result, meta)

cobra.CheckErr(err)


switch {

case  decResult.IsSuccess():

fmt.Printf("Execution successful.\n")

if  result != nil {

err = cbor.Unmarshal(decResult.Ok, result)

cobra.CheckErr(err)

}

default:

cobra.CheckErr(fmt.Sprintf("Execution failed with error: %s",

PrettyErrorHints(ctx, npa, conn, tx, meta, decResult.Failed)))

}

}
```

```
}
```

잔액 조회 시스템

계정 정보 조회 (`cmd/account/show/show.go`)

조회 과정

```
func showCmd() {
// 1. 대상 주소 결정
var targetAddress string
switch {
case len(args) >= 1:
targetAddress = args[0] // 명시적 주소
case npa.Account != nil:
targetAddress = npa.Account.Address // 기본 계정
default:
cobra.CheckErr("no address given and no wallet configured")
}

// 2. 네트워크 연결
ctx := context.Background()
c, err := connection.Connect(ctx, npa.Network)
cobra.CheckErr(err)
```

```
// 3. 주소 해석
nativeAddr, ethAddr, err := common.ResolveLocalAccountOrAddress(npa.Network, tar
cobra.CheckErr(err)

// 4. 블록 높이 조회
height, err := common.GetActualHeight(ctx, c.Consensus().Core())
cobra.CheckErr(err)

// 5. 합의 계층 계정 조회
ownerQuery := &staking.OwnerQuery{
Owner: nativeAddr.ConsensusAddress(),
Height: height,
}
consensusAccount, err := c.Consensus().Staking().Account(ctx, ownerQuery)
cobra.CheckErr(err)

// 6. ParaTime 계정 조회 (설정된 경우)
if  npa.ParaTime != nil {
rtBalances, err := c.Runtime(npa.ParaTime).Accounts.Balances(ctx, round, *native
cobra.CheckErr(err)

nonce, err := c.Runtime(npa.ParaTime).Accounts.Nonce(ctx, round, *nativeAddr)
```

```
cobra.CheckErr(err)

}

}
```

표시되는 정보

1. **기본 정보**

- 계정명 (있는 경우)
- 네이티브 주소
- 이더리움 주소 (있는 경우)

2. **합의 계층 정보**

- 논스 (nonce)
- 잔액 및 위임 정보
- 허용량 (allowances)
- 수수료 스케줄
- 스테이크 누적기

3. **ParaTime 정보**

- 논스
- 모든 토큰의 잔액
- 위임 및 언위임 정보

위임 정보 조회

위임 조회 과정

```
if  showDelegations {
// 나가는 위임 조회
outgoingDelegations, err := c.Consensus().Staking().DelegationInfosFor(ctx, owne
cobra.CheckErr(err)
// 나가는 언본딩 위임 조회
outgoingDebondingDelegations, err := c.Consensus().Staking().DebondingDelegation
cobra.CheckErr(err)


// 들어오는 위임 조회
incomingDelegations, err := c.Consensus().Staking().DelegationsTo(ctx, ownerQuer
cobra.CheckErr(err)
// 들어오는 언본딩 위임 조회
incomingDebondingDelegations, err := c.Consensus().Staking().DebondingDelegation
cobra.CheckErr(err)
}
```

트랜잭션 설정 및 플래그

공통 트랜잭션 플래그

런타임 트랜잭션 플래그

```
RuntimeTxFlags = flag.NewFlagSet("", flag.ContinueOnError)
```

```
RuntimeTxFlags.BoolVar(&txOffline, "offline", false, "do not perform any operati
RuntimeTxFlags.Uint64Var(&txNonce, "nonce", invalidNonce, "override nonce to use
RuntimeTxFlags.Uint64Var(&txGasLimit, "gas-limit", invalidGasLimit, "override ga
RuntimeTxFlags.StringVar(&txGasPrice, "gas-price", "", "override gas price to us
RuntimeTxFlags.StringVar(&txFeeDenom, "fee-denom", "", "override fee denominatio
RuntimeTxFlags.BoolVar(&txEncrypted, "encrypted", false, "encrypt transaction ca
RuntimeTxFlags.BoolVar(&txUnsigned, "unsigned", false, "do not sign transaction"
RuntimeTxFlags.StringVar(&txFormat, "format", "json", "transaction output format
RuntimeTxFlags.StringVarP(&txOutputFile, "output-file", "o", "", "output transac
```

합의 트랜잭션 플래그

```
TxFlags = flag.NewFlagSet("", flag.ContinueOnError)
TxFlags.BoolVar(&txOffline, "offline", false, "do not perform any operations req
TxFlags.Uint64Var(&txNonce, "nonce", invalidNonce, "override nonce to use")
TxFlags.Uint64Var(&txGasLimit, "gas-limit", invalidGasLimit, "override gas limit
TxFlags.StringVar(&txGasPrice, "gas-price", "", "override gas price to use")
TxFlags.BoolVar(&txUnsigned, "unsigned", false, "do not sign transaction")
TxFlags.StringVar(&txFormat, "format", "json", "transaction output format (for o
TxFlags.StringVarP(&txOutputFile, "output-file", "o", "", "output transaction in
```

트랜잭션 설정 구조

```
type TransactionConfig struct {
Offline  bool  // 오프라인 모드
Export  bool  // 내보내기 모드
}

func  GetTransactionConfig() *TransactionConfig {
return &TransactionConfig{
Offline: txOffline,
Export: shouldExportTransaction(),
}
}

func  shouldExportTransaction() bool {
return  txOffline || txUnsigned || txOutputFile != ""
}
```

에러 처리 및 검증

주소 검증

합의 계층 호환성 검증

```
func  CheckAddressIsConsensusCapable(cfg *config.Config, address  string) error
// 이더리움 주소 확인
```

```
if ethCommon.IsHexAddress(address) {
return fmt.Errorf("signer for address '%s' will not be able to sign transaction
}

// 지갑 계정 확인
for name, acc := range cfg.Wallet.All {
if acc.Address == address && !acc.HasConsensusSigner() {
return fmt.Errorf("destination account '%s' (%s) will not be able to sign trans
}
}

return nil
}
```

예약된 주소 검증

```
func CheckAddressNotReserved(cfg *config.Config, address string) error {
// ParaTime 풀 주소들
if address == rewards.RewardPoolAddress.String() {
return fmt.Errorf("address '%s' is ParaTime rewards pool address", address)
}
if address == accounts.CommonPoolAddress.String() {
return fmt.Errorf("address '%s' is ParaTime common pool address", address)
```

```
}

// 합의 계층 풀 주소들
if address == staking.BurnAddress.String() {
return fmt.Errorf("address '%s' is consensus burn address", address)
}

return nil
}
```

에러 힌트 시스템

```
func PrettyErrorHints(
_ context.Context,
npa *NPASelection,
_ connection.Connection,
_ interface{},
_ interface{},
failedRes *types.FailedCallResult,
) string {
errMsg := failedRes.Error()
// 테스트넷 토큰 부족 힌트
if npa != nil && npa.ParaTime != nil &&
```

```
npa.Network.ChainContext == config.DefaultNetworks.All["testnet"].ChainContext &
npa.ParaTime.ID == config.DefaultNetworks.All["testnet"].ParaTimes.All["sapphire
(failedRes.Module == accounts.ModuleName && failedRes.Code == 2 ||
failedRes.Module == core.ModuleName && failedRes.Code == 5) {
errMsg += "\nTip: You can get TEST tokens at https://faucet.testnet.oasis.io or
}
// 스테이킹 관련 힌트
if failedRes.Module == staking.ModuleName {
if failedRes.Code == 5 {
errMsg += "\nTip: Did you forget to run `oasis account allow`?"
} else if failedRes.Code == 9 {
errMsg += "\nTip: You can see minimum staking transfer amount by running `oasis
}
}
return errMsg
}
```

오프라인 모드 지원

오프라인 트랜잭션 생성

1. **논스 수동 지정**: `--nonce` 플래그로 논스 명시
2. **가스 한도 수동 지정**: `--gas-limit` 플래그로 가스 한도 명시

3. **가스 가격 수동 지정**: `--gas-price` 플래그로 가스 가격 명시
4. **서명되지 않은 트랜잭션**: `--unsigned` 플래그로 서명 생략
5. **파일 출력**: `--output-file` 플래그로 파일에 저장

트랜잭션 내보내기 형식

1. **JSON 형식**: 기본 형식, 사람이 읽기 쉬움
2. **CBOR 형식**: 바이너리 형식, 효율적인 저장

보안 고려사항

서명 전 확인

```
func PrintTransactionBeforeSigning(npa *NPASelection, tx interface{}) {
fmt.Printf("You are about to sign the following transaction:\n")
PrintTransaction(npa, tx)
fmt.Printf("Account: %s", npa.AccountName)
// 사용자 확인 요청
Confirm("Sign this transaction?", "signing aborted")
fmt.Println("(In case you are using a hardware-based signer you may need to conf
}
```

하드웨어 지갑 지원

1. **디바이스 확인**: 하드웨어 디바이스에서 트랜잭션 확인
2. **안전한 서명**: 개인키가 디바이스를 벗어나지 않음

3. **사용자 인터페이스**: 디바이스 확인 안내 메시지

성능 최적화

가스 추정

1. **자동 추정**: 온라인 모드에서 자동 가스 추정
2. **여유분 추가**: ParaTime 트랜잭션의 경우 20% 여유분 추가
3. **수동 지정**: 오프라인 모드에서 수동 지정 필요

연결 관리

1. **연결 재사용**: 동일한 네트워크에 대한 연결 재사용
2. **적절한 타임아웃**: 네트워크 상황에 맞는 타임아웃 설정
3. **에러 복구**: 연결 실패 시 재시도 로직

이 분석 문서는 Oasis CLI의 계정 관리 및 트랜잭션 처리 시스템의 전체적인 구조와 동작 방식을 상세히 설명하며, 개발자가 시스템을 이해하고 확장하는 데 필요한 모든 정보를 제공합니다.